

Manuale Tecnico – The-Knife

Versione: 1.0 • Data: 20/08/2025

Autori:

- ★ Francesco Girlanda 760616 VA
- ★ Mattia Lambertoni 762595 VA
- ★ Gabriele Gallon 761125 VA

Sommario

Introduzione.....	3
Librerie esterne utilizzate.....	3
Struttura generale del progetto.....	4
Classi.....	5
Dettaglio Dominio.....	7
○ Utente.....	7
○ Ristorante.....	7
○ Recensione-Preferito.....	7
○ Enumerativi.....	8
Dettaglio Interfacce.....	9
○ Identificabile.....	9
○ Card<T>.....	9
Dettaglio Utility.....	10
○ GestoreCSV <T> (astratta).....	10
○ Coordinate.....	13
○ Indirizzi.....	15
○ Criptatore.....	15
○ SceneManager.....	16
○ Gestione Eccezioni.....	17
○ BlockTeeStream.....	18
Dettaglio Controller (i più importanti).....	20
○ Login Controler.....	20
○ Aggiungi Ristorante.....	21
○ Aggiungi Recensione.....	22
Limiti della struttura.....	24
Bibliografia e riferimenti.....	26

Introduzione

The-Knife è un'applicazione Java con interfaccia **JavaFX** per la consultazione e gestione di ristoranti. I dati vengono memorizzati su **file CSV** e l'interfaccia è definita in **FXML** tramite l'utilizzo di **SceneBuilder**. La struttura del progetto è realizzata con **Maven**

Librerie esterne utilizzate

Libreria / Pacchetto	Scopo nel progetto	Classi che la usano (principali)
JavaFX (<code>javafx.*</code>)	GUI, FXML, scene, controlli grafici	Tutti i controller in <code>com.gruppo10.controller.*</code> , <code>SceneManager</code> , <code>GestioneEccezioni</code> , launcher <code>Main</code> e <code>TheKnife</code> in <code>fileJava/</code>
ControlsFX (<code>org.controlsfx.control.*</code>)	Auto-complete nei campi di testo	<code>AggiungiRistoranteController</code> , <code>LoginController</code> , <code>RegistrazioneController</code>
OpenCSV (<code>com.opencsv.*</code>)	Lettura/scrittura CSV, validazione	<code>GestoreCSV</code> , <code>GestionePreferiti</code> , <code>RecensioneCSV</code>
Gson (<code>com.google.gson.*</code>)	Parsing JSON per geocoding/autocomplete	<code>Coordinate</code> , <code>Indirizzi</code>
Lombok (<code>lombok.Data</code>)	Boilerplate (getter/setter/equals/hashCode)	<code>Coordinate</code> , <code>Recensione</code> , <code>Ristorante</code> , <code>Utente</code>

Nota: oltre alle librerie esterne sopra, sono usate API standard **Java 11+**: `java.net.http.HttpClient` per chiamate HTTP (geocoding/autocomplete OpenStreetMap), `java.security.MessageDigest` (hash password), `java.nio/java.io` per I/O.

Struttura generale del progetto

Abbiamo utilizzato Maven(v4.0.0) per strutturare il progetto

- **Cos'è Maven:** tool per build e gestione dipendenze Java (compila, testa, impacchetta, pubblica).
- **Project Object Model:** il cuore di Maven è il file pom.xml, per la gestione di groupId/artifactId/version, packaging, dipendenze, plugin e configurazioni.
 -
- **Dipendenze:** risolte da repo remoti → Maven ricerca la dipendenza dichiarate in repository remote e le scarica in una cache locale ~/.m2/repository, in modo che siano successivamente pronte all'uso.
- **Plugin & lifecycle:** i plugin eseguono "goal" agganciati alle fasi del progetto(compile, test, package, install, deploy).

Comandi e plugin utilizzati:

- [javafx-maven-plugin \(v0.0.8\)](#): permette di lanciare applicazioni JavaFX tramite il comando mvn javafx:run.
- [maven-compiler-plugin](#): compila il codice sorgente ed è configurabile per gestire, ad esempio, la libreria Lombok.
- [maven-javadoc-plugin](#): serve per generare la javadoc.
- [maven-shade-plugin](#): configurabile per creare un fat jar (incorpora direttamente anche le librerie esterne).
- [mvn clean](#): elimina la cartella target, contenente le classi compilate ed eventuali file .jar e/o javadoc
- [mvn package](#): maven esegue il ciclo di vita standard validate -> compile -> test -> package creando il file .jar finale
- [mvn javadoc:javadoc](#): tramite il plugin sopracitato genera la javadoc in formato html

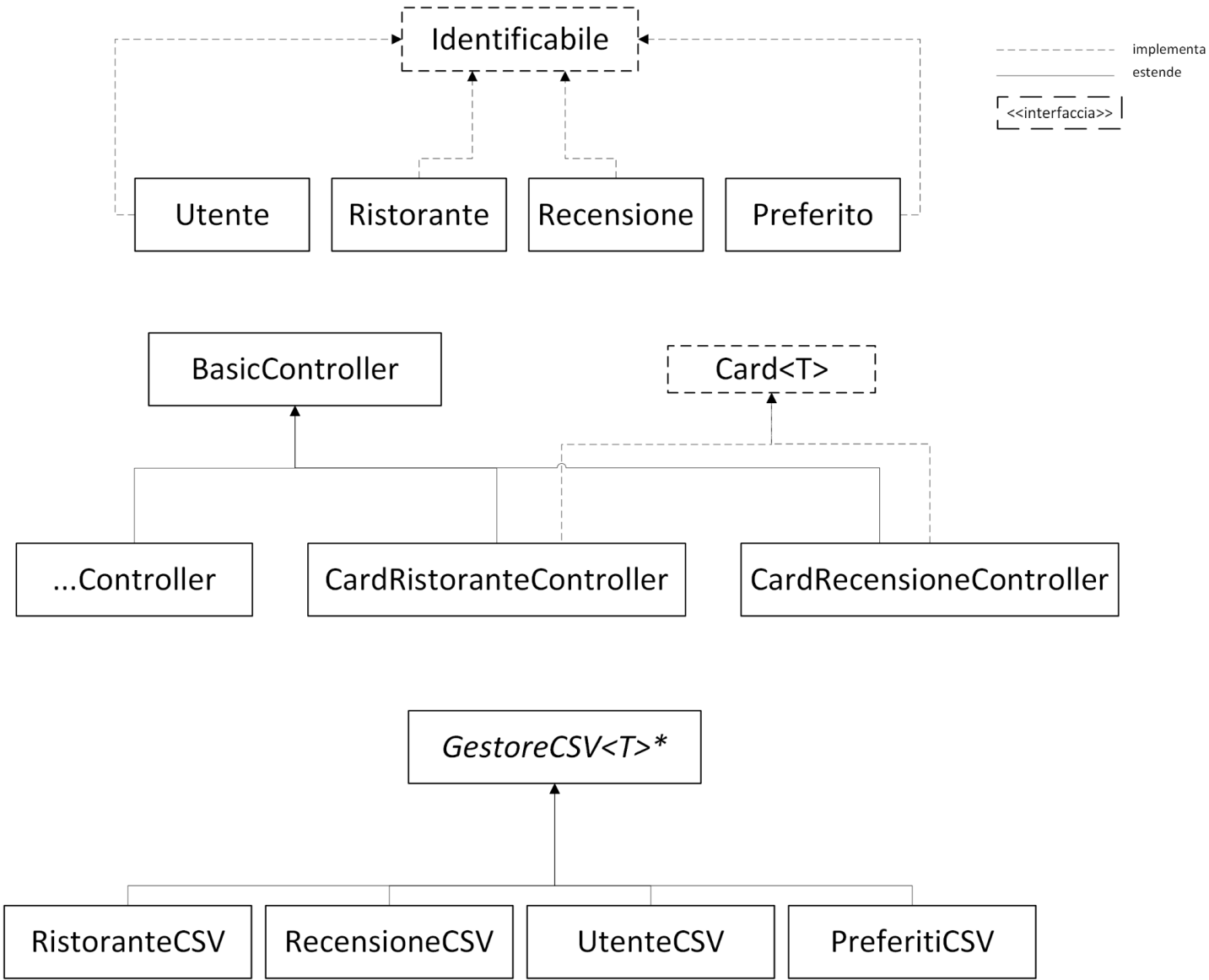
Esempio struttura progetto:

```
progetto/
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/           # codice
│   │   ├── resources/      # risorse incluse nel JAR
│   │   └── webapp/          # (solo WAR)
│   └── test/
│       ├── java/           # test
│       └── resources/
```

Classi

- **Dominio (Model)** – `com.gruppo10.classi`
 - Entità e tipi - Modellano oggetti utili per l'applicazione: `Ristorante`, `Utente`, `Recensione`, `Preferito`
 - Enumerativi - utilizzati principalmente per i filtri di ricerca nell'applicazione: `Prenotazione`, `Prezzo`, `TipoCucina`, `Delivery`, `Ruolo`, `Distanza`, `MediaRecensioni`.
- **Interfacce** – `com.gruppo10.classi/com.gruppo10.controller`
 - `Identificabile`
 - `Card`
- **Utility** – `com.gruppo10.classi`
 - `Coordinate`, `Indirizzi`, `SceneManager`, `GestioneEccezioni`, `GestoreCSV`, `Criptatore`, `BlockTeeStream`.
 - Mapper CSV: `RistoranteCSV`, `RecensioneCSV`, `UtenteCSV`.
- **Controller (GUI)** – `com.gruppo10.controller`
 - Astratta: `BasicController`
 - Specifici: `LoginController`, `PaginaPrincipaleController`, `ProfiloClienteController`, `ProfiloRistoratoreController`, `AggiungiRistoranteController`, ecc.
- **Entry point** – `com.gruppo10`
 - `Main`, `TheKnife` (entry point originale, il metodo main è stato spostato in una classe a parte per la generazione del fat jar)

Schema Gerarchia



Dettaglio Dominio

Utente

Responsabilità: modellare gli utenti (clienti, ristoratori, non registrati) all'interno del programma.

Ristorante

Responsabilità: modellare i ristoranti all'interno del programma e gestire il calcolo della media recensioni.

Metodi chiave:

```
public void aggiungiRecensione(Recensione recensione) {  
    this.recensioni.add(recensione);  
    int stelle = recensione.getStelle();  
    int tot = this.recensioni.size();  
    this.mediaRec = ((mediaRec * (tot - 1)) + stelle) / tot;  
}
```

Complessità: $O(1)$ (ricalcolo media senza scorrere tutte le recensioni)

```
public void rimuoviRecensione(Recensione recensione) {  
    this.recensioni.remove(recensione);  
    int tot = this.recensioni.size();  
    int stelle = recensione.getStelle();  
    if (tot == 0) {  
        this.mediaRec = 0.0;  
    } else {  
        this.mediaRec = (mediaRec * (tot + 1) - stelle) / tot;  
    }  
}
```

Complessità: $O(n)$ (rimozioni della recensione n = dimensione lista recensioni)

Recensione-Preferito

Responsabilità: modellare le recensioni e i preferiti all'interno del programma.

Enumerativi

Responsabilità: modellare alcune proprietà dei ristoranti.

Metodi chiave: eccetto **Ruolo** sovrascrivono il metodo `toString()` per restituire valori leggibili utilizzati nei filtri della ricerca.

Esempio su **TipoCucina**:

```
@Override
public String toString() {
    if (this == TipoCucina.TUTTO) {
        return this.name();
    }
    String cucina = this.name()
                    .toLowerCase()
                    .replace('_', ' ');
    return cucina.substring(0, 1).toUpperCase() + cucina.substring(1);
}
```


Dettaglio Interfacce

Identificabile

Obiettivo: permettere di generalizzare la rimozione dal file csv di recensioni e preferiti, utilizzata anche per ristoranti e utenti per comodità e coerenza logica.

Responsabilità: modellare il comportamento delle classi modello di cui sopra.

Implementazioni: `Ristorante`, `Utente`, `Recensione`, `Preferito`.

Card<T>

Obiettivo: generalizzare il caricamento delle tessere, in particolare tramite `setItem`, usato per settare l'oggetto in un metodo generico `caricaTessere`, presente nell'utility `SceneManager`, che accetta sia oggetti di tipo `Ristoratore` che di tipo `Recensione`.

Responsabilità: modellare il comportamento dei controller di tipo Card.

Implementazioni: `CardRistoratoreController`, `CardRecensioneController`.

Metodi chiave:

```
void setStage(Stage stage);  
void setDati();  
void setItem(T item, VBox contenitore);
```

Dettaglio Utility

GestoreCSV <T> (astratta)

Responsabilità: Incapsula l'I/O su CSV per un tipo T (lettura, scrittura e metodi generici).

Estensioni: RistoranteCSV, RecensioneCSV, UtenteCSV implementano parseRiga/estraiDati/getHeader/extra.

Strutture Dati: ArrayList per oggetti da scrivere/leggere. Array di stringhe per l'estrazione di dati da csv o da oggetto.

Metodi chiave:

```
public List<T> caricaCSV() {
    List<T> lista = new ArrayList<>();
    caricamentoExtra();
    try (CSVReader reader = new CSVReader(new FileReader(file))) {
        String[] dati;
        reader.readNext(); // Salta l'header
        while ((dati = reader.readNext()) != null) {
            T obj = parseRiga(dati);
            lista.add(obj);
        }
    } catch (CsvValidationException e) {
        GestioneEccezioni.errore(
            "Errore format csv in: " + file,
            e,
            true,
            nuovoFile -> file = nuovoFile
        );
        return null;
    } catch (IOException e) {
        if (!(e instanceof FileNotFoundException)) {
            GestioneEccezioni.errore(
                "Errore caricamento file: " + file,
                e,
                true,
                nuovoFile -> file = nuovoFile
            );
            return null;
        }
    }
    return lista;
}
```

Complessità: $O(C+n \cdot P)$ dove C = complessità caricamento extra, n = numero di righe nel csv, P = complessità di parseRiga

```
public void scrivi(T obj) {
    if (!dir.exists()) {
        if (!dir.mkdirs()) {
            GestioneEccezioni.errore(
                "Impossibile creare la directory fileCSV",
                null,
                false,
                null
            );
        }
    }
    boolean fileEsiste = file.exists();
    try (Writer writer = new FileWriter(file, true);
        CSVWriter csvWriter = new CSVWriter(writer)) {
        if (!fileEsiste) {
            csvWriter.writeNext(getHeader());
            csvWriter.flush();
        }
        String[] dati = estraiDati(obj);
        csvWriter.writeNext(dati);
        csvWriter.flush();
    } catch (IOException e) {
        if (!(e instanceof FileNotFoundException)) {
            GestioneEccezioni.errore(
                "Errore caricamento file: " + file,
                e,
                true,
                nuovoFile -> file = nuovoFile
            );
        }
    } catch (SecurityException e) {
        GestioneEccezioni.errore(
            "Permesso negato per la scrittura del file: " + file,
            e,
            true,
            nuovoFile -> file = nuovoFile
        );
    } catch (IllegalArgumentException e) {
        GestioneEccezioni.errore(
            "Errore nei dati della recensione",
            e,
            false,
            null
        );
    }
}
```

```
}
```

Complessità: $O(1)$ (estraiDati ha sempre complessità $O(1)$)

```
public void rimuovi(T obj) {
    List<String[]> listaTemp = new ArrayList<>();
    try (CSVReader reader = new CSVReader(new FileReader(file))) {
        String[] dati;
        reader.readNext(); // Salta l'header
        while ((dati = reader.readNext()) != null) {
            try {
                int idUt = Integer.parseInt(dati[0]);
                int idRis = Integer.parseInt(dati[1]);
                if (!(idUt == obj.getIdUtente() && idRis == obj.getIdRistorante())) {
                    listaTemp.add(dati);
                }
            } catch (NumberFormatException e) {
                GestioneEccezioni.errore(
                    "Errore parsing ID\nRiga: " + Arrays.toString(dati),
                    e,
                    false,
                    null
                );
            }
        }
    } catch (IOException e) {
        GestioneEccezioni.errore(
            "Errore caricamento file: " + file,
            e,
            true,
            nuovoFile -> file = nuovoFile
        );
        return;
    } catch (CsvValidationException e) {
        GestioneEccezioni.errore(
            "Errore di validazione CSV: " + file,
            e,
            true,
            nuovoFile -> file = nuovoFile
        );
        return;
    }
    sovrascrivi(listaTemp);
}
```

Complessità: $O(n)$ (sovrascrivi ha sempre complessità $O(n)$ dove n = numero di righe da scrivere)

Coordinate

Obbiettivo: rappresentare un punto geografico (latitudine e longitudine) e fornire funzionalità di geocoding per il calcolo delle distanze.

Responsabilità: Modellare un punto geografico. Utilizza geocoding stringa indirizzo → lat/long usando **Nominatim (OpenStreetMap)** via **HttpClient**, esegue il parsing JSON con **Gson** e si occupa del calcolo delle distanze tramite formula Haversine.

Metodi chiave:

```
public Coordinate(String indirizzo) {
    String encodedIndirizzo = indirizzo.replace(" ", "+");
    String url = "https://nominatim.openstreetmap.org/search?q=" + encodedIndirizzo +
"&format=json&limit=1";
    HttpClient client = HttpClient.newHttpClient();
    HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create(url))
        .header("User-Agent", "JavaFXApp/1.0")
        .build();
    try {
        HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());
        if (response.statusCode() != 200) {
            GestioneEccezioni.errore(
                "Errore HTTP\nStatus code: " + response.statusCode(),
                null,
                false,
                null
            );
            this.lat = null;
            return;
        }
        JsonArray results = JsonParser.parseString(response.body()).getAsJsonArray();
        if (results.size() == 0) {
            GestioneEccezioni.errore(
                "Errore calcolo coordinate\nNessun risultato trovato per: " +
indirizzo,
                null,
                false,
                null
            );
            this.lat = null;
            return;
        }
    }
```

```

        JsonObject obj = results.get(0).getAsJsonObject();
        double lat = obj.get("lat").getAsDouble();
        double lon = obj.get("lon").getAsDouble();

        this.lat = lat;
        this.lon = lon;
    } catch (IOException e) {
        GestioneEccezioni.errore(
            "Errore di rete durante la richiesta HTTP",
            e,
            false,
            null
        );
        this.lat = null;
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        GestioneEccezioni.errore(
            "Errore: richiesta interrotta",
            e,
            false,
            null
        );
        this.lat = null;
    } catch (JsonSyntaxException e) {
        GestioneEccezioni.errore(
            "Errore nel parsing della risposta JSON",
            e,
            false,
            null
        );
        this.lat = null;
    }
}

```

Complessità: $O(L)$ dove L = lunghezza della query (coinvolgendo chiamate HTTP, la velocità della connessione può essere determinante per il tempo di calcolo)

```

public double calcolaDistanza(Coordinate c2) {
    double deltaLat = Math.toRadians(c2.getLat() - this.getLat());
    double deltaLon = Math.toRadians(c2.getLon() - this.getLon());
    double a = Math.sin(deltaLat / 2) * Math.sin(deltaLat / 2) +
        Math.cos(Math.toRadians(this.getLat())) *
        Math.cos(Math.toRadians(c2.getLat())) *
        Math.sin(deltaLon / 2) * Math.sin(deltaLon / 2);

    double c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
    return R * c; // Distanza in km
}

```

```
}
```

Complessità: $O(1)$

Indirizzi

Obbiettivo: fornire suggerimenti di indirizzi tramite autocompletamento interrogando il servizio di geocoding

Responsabilità: suggerimenti autocompletamento indirizzi (5 risultati max) interrogando **Nominatim**; parsing con **Gson**. Restituisce **List<String>**.

Strutture Dati: ArrayList per risultati trovati, dimensione variabile.

Metodi chiave: geocoding analogo a **Coordinate**.

Criptatore

Obbiettivo: fornire un metodo semplice per convertire stringhe in hash non reversibili tramite l'algoritmo crittografico SHA-256.

Responsabilità: hashing credenziali con **SHA-256** (API standard).

Metodi chiave:

```
public static String cripta(String input) {
    try {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hashBytes = digest.digest(input.getBytes());
        StringBuilder hexString = new StringBuilder();
        for (byte b : hashBytes) {
            hexString.append(String.format("%02x", b));
        }
        return hexString.toString();
    } catch (NoSuchAlgorithmException e) {
        GestioneEccezioni.errore(
            "Algoritmo SHA-256 non disponibile",
            e,
            false,
            null
        );
        return null;
    }
}
```

Complessità: $O(1)$

SceneManager

Obiettivo: factory di scene/controller con callback `controllerConsumer` per passare dipendenze/dati al controller, chiamando eventuali funzioni specifiche.

Responsabilità: centralizzare il cambio scena/apertura finestre dell'applicazione.

Metodi chiave:

```
public static <T> void caricaTessere(  
    List<T> lista,  
    VBox contenitoreTessere,  
    Stage stage,  
    String fxmlFile,  
    BiConsumer<Card<T>, T> extraConfig  
) {  
    contenitoreTessere.getChildren().clear();  
    for (T r : lista) {  
        try {  
            FXMLLoader loader = new FXMLLoader(TheKnife.class.getResource(guiPath +  
fxmlFile));  
            HBox card = loader.load();  
            Card<T> controller = loader.getController();  
            controller.setStage(stage);  
            if (extraConfig != null) {  
                extraConfig.accept(controller, r);  
            }  
            controller.setItem(r, contenitoreTessere);  
            controller.setDati();  
            contenitoreTessere.getChildren().add(card);  
        } catch (IOException e) {  
            GestioneEccezioni.errore(  
                "Errore caricamento file: " + fxmlFile,  
                e,  
                false,  
                null  
            );  
            return;  
        }  
    }  
}
```

Complessità: $O(n)$ dove n = dimensione della lista

```
public static <T> void cambioScena(  
    Stage stage,  
    String fxmlFile,  
    String title,
```



```

        Consumer<T> controllerConsumer
    ) {
        try {
            FXMLLoader loader = new FXMLLoader(TheKnife.class.getResource(guiPath +
fxmlFile));
            Parent root = loader.load();
            Scene scene = new Scene(root);

            stage.setScene(scene);
            stage.setTitle(title);
            stage.show();
            T controller = loader.getController();
            if (controllerConsumer != null) {
                controllerConsumer.accept(controller);
            }
        } catch (IOException e) {
            GestioneEccezioni.errore(
                "Errore caricamento file: " + fxmlFile,
                e,
                false,
                null
            );
        }
    }
}

```

Complessità: $O(1)$ (il rendering della GUI può dominare il tempo di esecuzione)

Gestione Eccezioni

Obiettivo: Notificare l'utente in caso di errore, senza bloccare il flusso del programma ed evitando di eseguire operazioni su dati compromessi dall'eccezione.

Responsabilità: Mostra **Alert** JavaFX con dettaglio stacktrace; opzionalmente propone un bottone "Scegli Altro..." per reindirizzare a un file CSV alternativo.

Metodi chiave:

```

public static void errore(
    String header,
    Exception e,
    boolean bottone,
    Consumer<File> azione
) {
    Alert alert = new Alert(Alert.AlertType.ERROR);
    alert.setTitle("Errore");
    alert.setHeaderText(header);
    if (e != null) {
        StringWriter sw = new StringWriter();
        PrintWriter pw = new PrintWriter(sw);
        e.printStackTrace(pw);
    }
}

```

```

        TextArea textArea = new TextArea(sw.toString());
        textArea.setEditable(false);
        textArea.setWrapText(true);
        alert.getDialogPane().setExpandableContent(textArea);
    }
    ButtonType scegliAltro = new ButtonType("Scegli un altro file");
    if (bottone) {
        alert.getButtonTypes().add(scegliAltro);
    }
    Optional<ButtonType> result = alert.showAndWait();
    if (result.isPresent() && result.get() == scegliAltro) {
        FileChooser fileChooser = new FileChooser();
        fileChooser.setTitle("Scegli un file CSV");
        File file = fileChooser.showOpenDialog(null);
        if (file != null) {
            azione.accept(file);
        }
    }
}

```

Complessità: nominalmente $O(s)$ dove s = dimensione stacktrace (la presenza di `showAndWait` ed eventuale azione di I/O per selezionare un nuovo rendono il tempo di esecuzione non deterministico)

BlockTeeStream

Obiettivo: Permettere logging ordinato e leggibile, mantenendo sincronizzati console e file con timestamp solo sui blocchi.

Responsabilità: Duplica l'output su console e file, aggiunge timestamp all'inizio di ogni blocco e fa flush immediato.

Metodi chiave:

```

@Override
public void write(int b) throws IOException {
    if (b == '\n') {
        if (buffer.length() > 0) {
            String line;
            if (isNewBlock) {
                String ts = LocalDateTime.now().format(fmt);
                line = "[" + ts + "] " + buffer + System.lineSeparator();
                isNewBlock = false;
            } else {
                line = buffer + System.lineSeparator();
            }
            byte[] bytes = line.getBytes();
            console.write(bytes);
        }
    }
}

```

```
console.flush();  
file.write(bytes);  
file.flush();  
buffer.setLength(0);  
} else {  
isNewBlock = true;  
}  
} else {  
buffer.append((char) b);  
}
```

Complessità: $O(N)$ perché ogni byte viene letto e scritto una sola volta.

Dettaglio Controller (i più importanti)

Login Controller

Obiettivo: Consentire l'accesso sicuro all'app (utente registrato o non registrato) e instradare l'utente alla pagina principale con UI reattiva.

Responsabilità: Gestisce la schermata di login JavaFX: validazione campi, autenticazione via CSV, accesso "ospite", navigazione a registrazione/pagina principale.

Metodi chiave:

@FXML

```
public void provaLogin() {
    String username = usernameField.getText();
    String password = passwordField.getText();
    String hashedPassword = Criptatore.cripta(password);

    if (hashedPassword == null)
        return;

    UtenteCSV utenteCSV = new UtenteCSV();
    Utente utente = utenteCSV.cercaUtente(username);
    if (utente == null) {
        loginStatus.setVisible(true);
        loginStatus.setText("UTENTE NON REGISTRATO");
        return;
    }

    if (hashedPassword.equals(utente.getPassword())) {
        utenteLoggato = utente;
        apriPaginaPrincipale();
    } else {
        loginStatus.setVisible(true);
        loginStatus.setText("PASSWORD ERRATA");
    }
}
```

Complessità: Tempo: $O(L + n)$ dove L è la lunghezza password (hashing) ed n è numero di utenti nel CSV (ricerca lineare).

@FXML

```
public void continuaSenzaRegistrarti() {
    String indirizzo = indirizzoField.getText();
    Coordinate coordinate = new Coordinate(indirizzo);
    if (coordinate.getLat() == null)
        return;
}
```

```

    utenteLoggato = new Utente();
    utenteLoggato.setRuolo("NON_REGISTRATO");
    utenteLoggato.setIndirizzo(indirizzo);
    utenteLoggato.setCords(coordinate);

    apriPaginaPrincipale();
}

```

Complessità: $O(1)$

Aggiungi Ristorante

Obiettivo: Permettere all'utente proprietario loggato di registrare un nuovo ristorante con tutte le informazioni (nome, indirizzo, prezzo, cucina, delivery, prenotazioni)

Responsabilità: Gestisce la vista per l'aggiunta di un nuovo ristorante: validazione campi, costruzione oggetto ristorante, salvataggio su CSV e notifica al chiamante.

Metodi chiave:

@FXML

```

private void aggiungiRistorante() throws Exception {

    RistoranteCSV ristoranteCSV = new RistoranteCSV();
    String indirizzo = indirizzoField.getText();
    Coordinate cords = new Coordinate(indirizzo);

    if (cords.getLat() == null) {
        return;
    }

    Delivery selectedDelivery = (Delivery) deliveryGroup
        .getSelectedToggle().getUserData();
    Prenotazione selectedPrenotazione = (Prenotazione) prenotazioneGroup
        .getSelectedToggle().getUserData();
    Prezzo selectedPrezzo = (Prezzo) prezzoGroup
        .getSelectedToggle().getUserData();

    String nomeRistorante = nomeRistoranteField.getText();
    int idProprietario = utenteLoggato.getId();

    Ristorante ristorante = new Ristorante();
    ristorante.setIdproprietario(idProprietario);
    ristorante.setNomeRistorante(nomeRistorante);
    ristorante.setIndirizzo(indirizzo);
    ristorante.setDelivery(selectedDelivery);
    ristorante.setPrenotazioneOnline(selectedPrenotazione);
    ristorante.setPrezzo(selectedPrezzo);
    ristorante.setTipoCucina(comboCucina.getValue());
}

```

```

    ristorante.setDescrizione(txtDescrizione.getText());
    ristorante.setCords(cords);

    this.nuovoRistorante = ristorante;

    try {
        ristoranteCSV.aggiungiRistorante(ristorante);
        ristoranteCSV.scrivi(ristorante);
    } catch (Exception e) {
        return;
    }

    if (onCloseCallback != null) {
        onCloseCallback.run();
    }

    chiudi();
}

```

Complessità: $O(n)$ dove n è il numero di ristoranti già memorizzati sul file csv.

Aggiungi Recensione

Obiettivo: Consentire all'utente loggato di aggiungere una recensione completa (testo + rating) a un ristorante.

Responsabilità: Gestisce la finestra di inserimento recensione: input testo, scelta stelle, validazione, creazione Recensione, salvataggio su CSV e refresh della pagina ristorante.

Metodi chiave:

@FXML

```

private void aggiungiRecensione() {
    String testo = txtTesto.getText();
    RadioButton selectedStella = (RadioButton) stelleGroup.getSelectedToggle();
    int stelle = selectedStella.getText().length();

    Recensione recensione = new Recensione();
    recensione.setUsername(utenteLoggato.getUsername());
    recensione.setIdUtente(utenteLoggato.getId());
    recensione.setIdRistorante(ristorante.getId());
    recensione.setStelle(stelle);
    recensione.setTesto(testo);
    recensione.setRisposta("");
    recensione.setRistorante(ristorante);

    ristorante.aggiungiRecensione(recensione);
}

```

```
try {
    RecensioneCSV recensioneCSV = new RecensioneCSV();
    recensioneCSV.scrivi(recensione);
} catch (Exception e) {
    return;
}

SceneManager.apriPaginaRistorante(stage, ristorante, paginaPrincipale,
indiceTab);
chiudi();
}
```

Complessità: $O(n)$ perché la persistenza su CSV è lineare nel numero di recensioni (n)

Limiti della struttura

Nonostante la soluzione proposta risulti funzionale e adatta a un contesto didattico o a un'applicazione su scala ridotta, presenta alcuni limiti legati alle scelte progettuali:

1. Memorizzazione dati su file CSV

- L'uso di file CSV, invece di un database relazionale o NoSQL, comporta diversi vincoli:
 - **Scalabilità limitata:** le operazioni di lettura/scrittura hanno complessità lineare rispetto al numero di righe. Questo significa che, con dataset di grandi dimensioni le prestazioni degradano sensibilmente.
 - **Ridotta flessibilità nelle query:** i CSV non permettono interrogazioni complesse come ad esempio ricerche multi-criterio ottimizzate che richiederebbero invece indici o linguaggi come SQL. Questa mancanza rende le ricerche più complicate sia a livello progettuale sia a livello di dispendio di risorse. Ne è un esempio la ricerca tramite filtri che nel nostro progetto viene effettuata concatenando numerosi AND, che usando un database sarebbe stata sostituita da una query più semplice da ideare e realizzare.
- Per questo motivo la soluzione è valida principalmente per dataset di piccole/medie dimensioni e per un utilizzo prettamente didattico

2. Dipendenza dalla connessione a Internet per alcune funzionalità

- Alcune funzionalità dell'applicazione, in particolare la gestione degli indirizzi e delle coordinate, sfruttano il servizio di geocoding/autocompletamento offerto da Nominatim OpenStreetMap.
- Ciò implica che:
 - In assenza di connessione a Internet queste funzionalità **non sono disponibili**, limitando l'esperienza d'uso, ad esempio non è possibile calcolare le distanze o suggerire indirizzi.
 - Eventuali **problemi di latenza o indisponibilità del servizio esterno** impattano direttamente l'applicazione, rendendo i tempi di risposta poco prevedibili.

- Non è previsto alcun **caching locale** dei risultati: ogni richiesta richiede una nuova chiamata HTTP.
- Questo vincolo può essere tollerabile in contesti didattici, ma non sarebbe accettabile in un sistema destinato a un uso professionale.

Bibliografia e riferimenti

- JavaFX (FXML, Controls): [*OpenJFX docs*](#)
- Maven: [*Maven Guides*](#)
- OpenCSV: [*opencsv user guide*](#)
- Gson: [*Gson javadoc*](#)
- OpenStreetMap Nominatim: [*manual*](#)